FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	John Louis M. Panganiban
Section:	AM41
Professor:	Ms. Crisola Tan
Date:	9/29/26

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

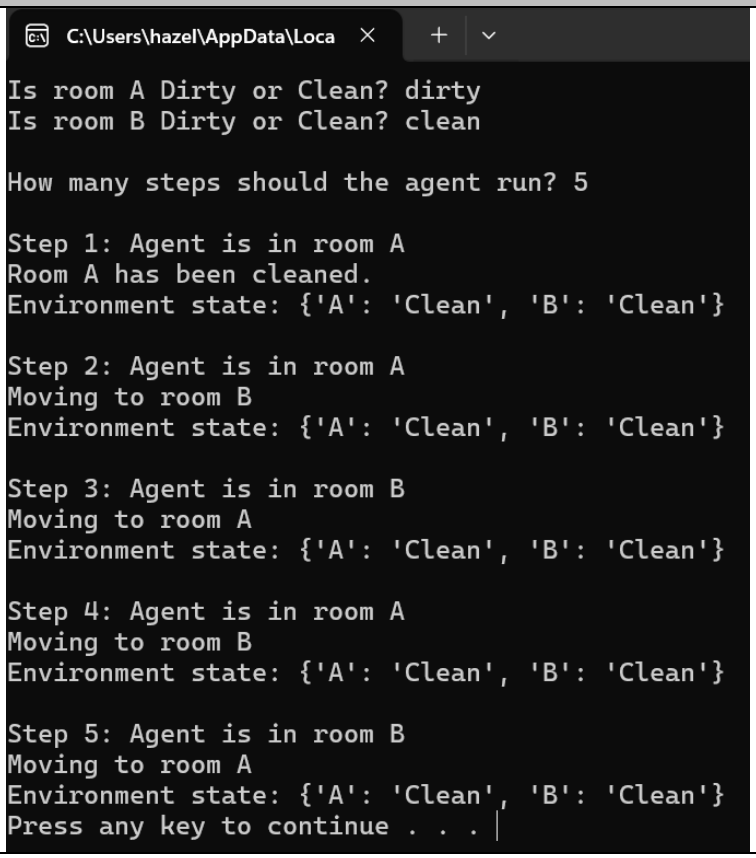
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

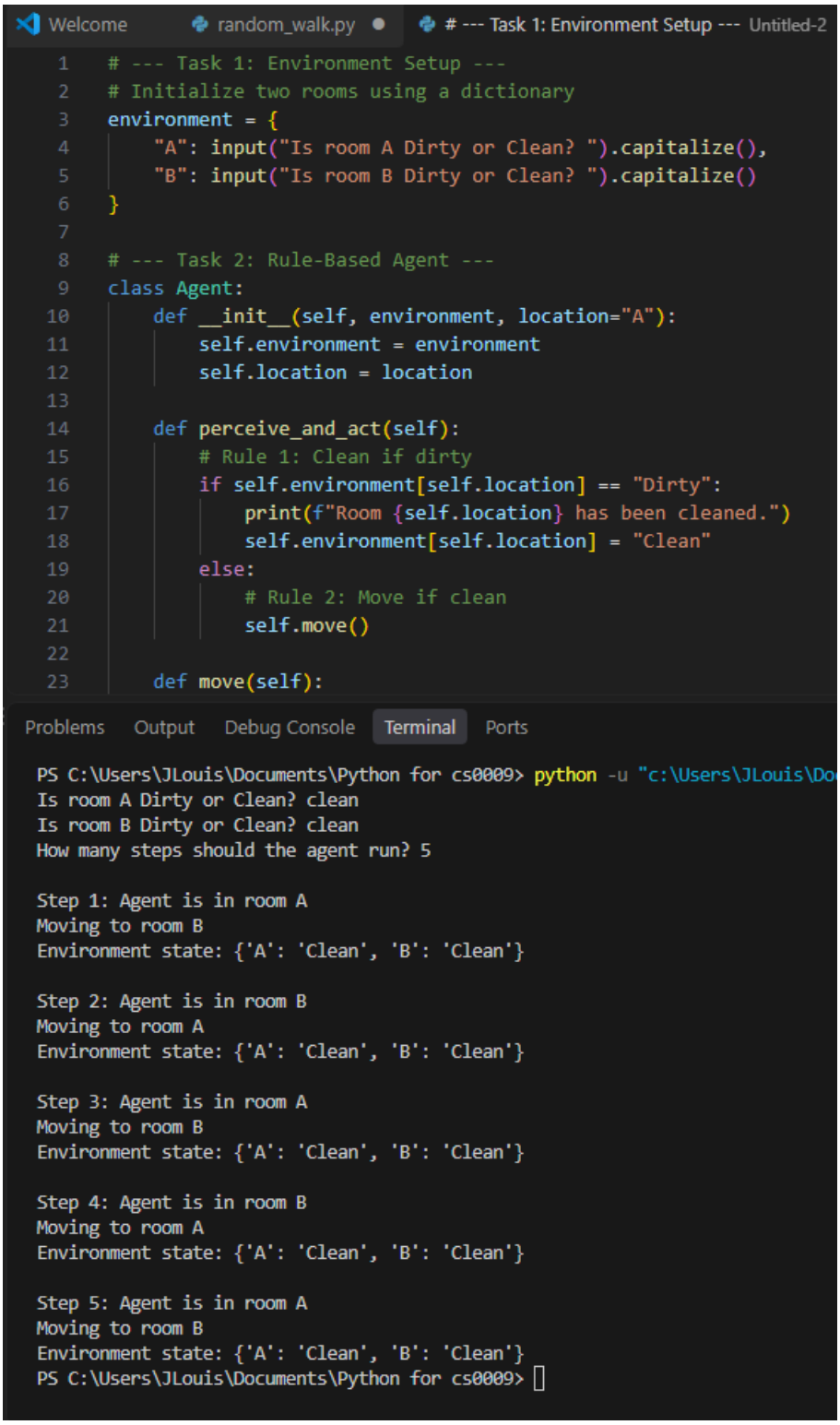
Sample Output





Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
    - Rule 1: If the current room is dirty, clean it.
    - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.



Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.

2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

```python
# --- Task 1: Environment Setup ---
environment = {
    "A": input("Is room A Dirty or Clean? ").capitalize(),
    "B": input("Is room B Dirty or Clean? ").capitalize()
}

# --- Task 2: Rule-Based Agent ---
class Agent:
    def __init__(self, environment, location="A"):
        self.environment = environment
        self.location = location

    def perceive_and_act(self):
        if self.environment[self.location] == "Dirty":
            print(f"Room {self.location} has been cleaned.")
            self.environment[self.location] = "Clean"
        else:
            self.move()

    def move(self):
        self.location = "B" if self.location == "A" else "A"
        print(f"Moving to room {self.location}")

```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\JLouis\Documents\Python for cs0009> python -u "c:\Users\JLouis\Docume
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean
How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

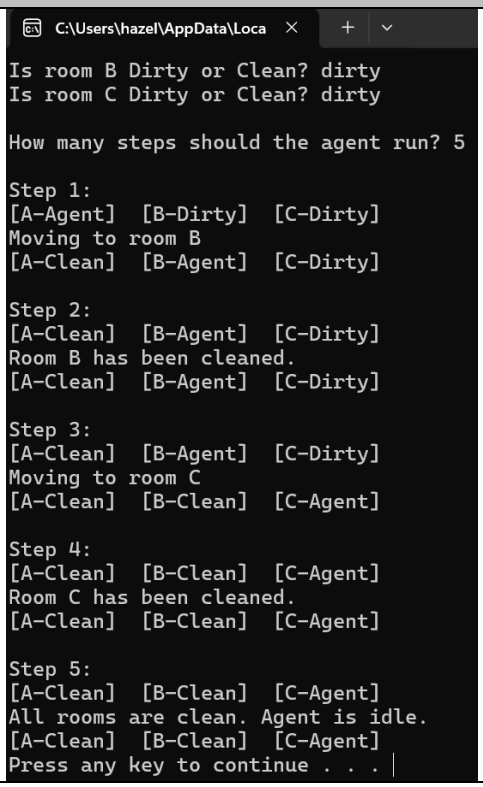
Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

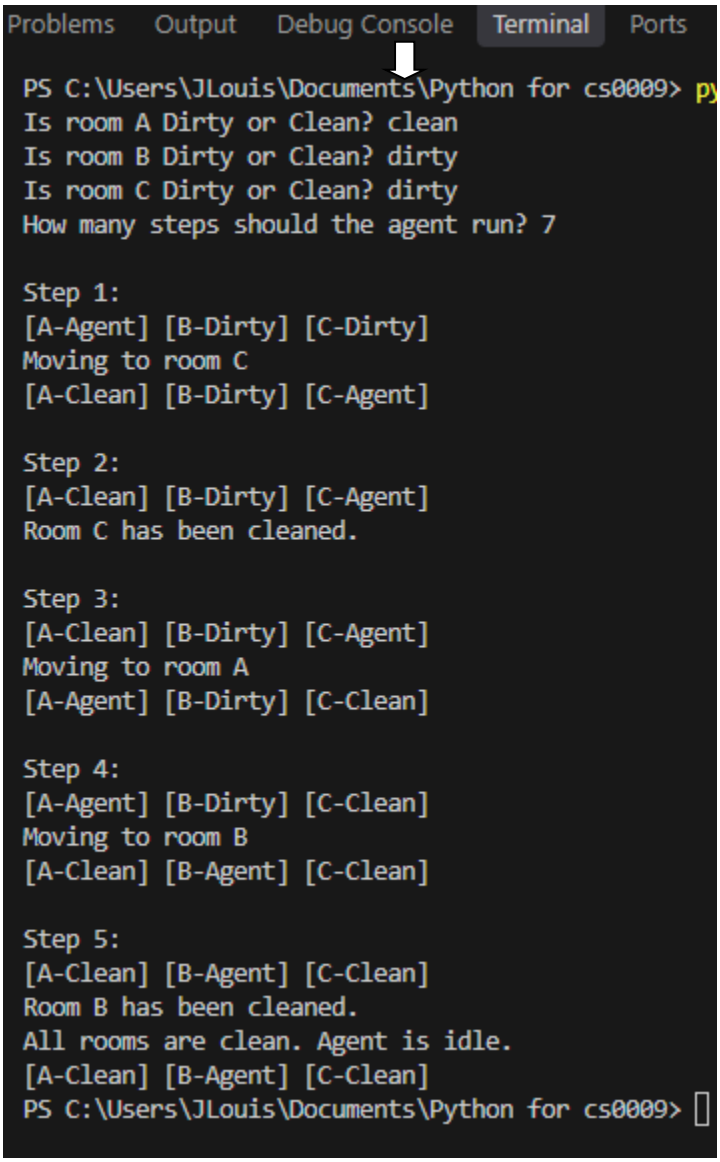
Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
PS C:\Users\JLouis\Documents\Python for cs0009>
```

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)





Your Code - Laboratory Tasks

Task 1 - 3

```
# --- Task 1: Environment Setup ---
# Initialize two rooms using a dictionary
environment = {
    "A": input("Is room A Dirty or Clean? ").capitalize(),
    "B": input("Is room B Dirty or Clean? ").capitalize()
}
```

```
# --- Task 2: Rule-Based Agent ---
class Agent:
    def __init__(self, environment, location="A"):
        self.environment = environment
        self.location = location

    def perceive_and_act(self):
        # Rule 1: Clean if dirty
        if self.environment[self.location] == "Dirty":
            print(f"Room {self.location} has been cleaned.")
            self.environment[self.location] = "Clean"
        else:
            # Rule 2: Move if clean
            self.move()

    def move(self):
        # Switch between rooms
        self.location = "B" if self.location == "A" else "A"
        print(f"Moving to room {self.location}")

# --- Task 3: Run the Simulation ---
steps = int(input("How many steps should the agent run? "))
agent = Agent(environment)

for step in range(1, steps + 1):
    print(f"\nStep {step}: Agent is in room {agent.location}")
    agent.perceive_and_act()
    print("Environment state:", environment)
```

Bonus Task:

```
import random

# --- Environment Setup ---
environment = {
    "A": input("Is room A Dirty or Clean? ").capitalize(),
    "B": input("Is room B Dirty or Clean? ").capitalize(),
    "C": input("Is room C Dirty or Clean? ").capitalize()
}

class Agent:
    def __init__(self, environment, location="A"):
        self.environment = environment
        self.location = location
```

```
    def perceive_and_act(self):
        # Display current grid
        self.display_environment()

        if self.environment[self.location] == "Dirty":
            print(f"Room {self.location} has been cleaned.")
            self.environment[self.location] = "Clean"
        else:
            self.move()

    def move(self):
        # Randomly choose another room
        possible_rooms = list(self.environment.keys())
        possible_rooms.remove(self.location)
        self.location = random.choice(possible_rooms)
        print(f"Moving to room {self.location}")
        self.display_environment()

    def display_environment(self):
        # Text-based grid visualization
        grid = []
        for room, state in self.environment.items():
            if room == self.location:
                grid.append(f"[{room}-Agent]")
            else:
                grid.append(f"[{room}-{state}]")
        print(" ".join(grid))

# --- Simulation Runner ---
steps = int(input("How many steps should the agent run? "))
agent = Agent(environment)

for step in range(1, steps + 1):
    print(f"\nStep {step}:")
    agent.perceive_and_act()

    # Check if all rooms are clean
    if all(state == "Clean" for state in environment.values()):
        print("All rooms are clean. Agent is idle.")
        agent.display_environment()
        break
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows two simple rules: if the current room is dirty, it cleans it; if the current room is clean, it moves to the other room. These rules are predefined and do not change during the simulation. The agent's behavior is entirely determined by these fixed conditions.

2. The agent perceives the state of the room it is currently in. If the perception shows "Dirty," it executes the cleaning action; otherwise, it executes the move action. This decision-making is based solely on the environment's state at that moment.

3. If both rooms are clean initially, the agent will never perform a cleaning action. Instead, it will continuously move back and forth between the two rooms. The environment state remains unchanged throughout the simulation.

4. To handle more than two rooms, you would expand the environment dictionary to include additional rooms. The agent's move logic could be updated to either cycle through rooms sequentially or randomly choose a new room. This modification allows the agent to scale its behavior to larger environments.